

REDUX TOOLKIT (RTK) vs ZUSTAND

Page 1

BASICS COMPARISON

★ 1. What are they?

- RTK → Official, opinionated, batteries-included state management for large-scale apps.
- Zustand → Small, simple, unopinionated store for React. Uses hook-based API.

Both
Manage
State!



★ 2. Core Philosophy

<u>Redux Toolkit (RTK)</u>	<u>Zustand</u>
Follow structure and patterns Best for predictability and maintainability.	Minimal API Do what you need, no extra rules.

★ 3. Boilerplate Code

RTK → More setup

- Store
- Slice
- Actions
- Provider

Zustand → Very less setup

- Create store
- Use store in component

★ 4. Setup Example

RTK Setup Flow

```
createSlice()
  ↓
configureStore()
  ↓
<Provider store={store}>
  ↓
useSelector / useDispatch()
```

Zustand Setup Flow

```
create()
  ↓
useStore()
  ↓
Done ✓
```

★ 5. State Access in Components

RTK

```
const value = useSelector(
  (state) => state.slice.value
)
```

```
const dispatch = useDispatch()
```

Zustand

```
const value = useStore(
  (state) => state.value
)
```

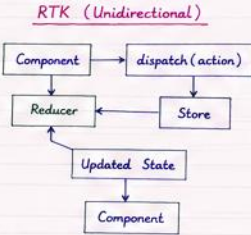
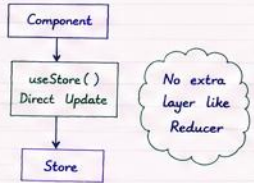
Direct hook usage!
No dispatch needed
for simple updates.



★ 6. State Update

<u>RTK</u>	<u>Zustand</u>
<code>dispatch(increment())</code> <code>dispatch(setValue(10))</code>	<code>const increment =</code> <code>useStore((state) → state.increment)</code> <code>increment()</code>
Updates are done by dispatching actions.	Call function directly from the store.

★ 7. Data Flow

Zustand

★ 8. Best For

<u>RTK</u>	<u>Zustand</u>
Large applications	Small to medium applications
Complex state logic	Simple state logic
Teams who like structure	Teams who like simplicity and flexibility
Need devtools, middleware, RTK Query, etc.	Need lightweight state management

★ 9. Middleware & DevTools

<u>RTK</u>	<u>Zustand</u>
✓ Built-in support	✓ Devtools middleware
✓ Rtk Toolkit DevTools	✓ Persist middleware
✓ Middleware ecosystem	✓ Very flexible

★ 10. Learning Curve

<u>RTK</u>
Medium

Zustand is easier for beginners.

<u>Zustand</u>
Easy

★ 11. Performance

Both are fast ⚡ but Zustand is slightly lighter (smaller bundle size).



★ 12. Feature Comparison

<u>Feature</u>	<u>RTK</u>	<u>Zustand</u>
State Management	Yes	Yes
Boilerplate	More	Very Less
Setup	Complex	Simple
DevTools	Built-in	Middleware
Middleware	Built-in support	Manual add
Async Logic (RTK Query)	Yes	No (Manual)
Persist State	Yes	Yes (Middleware)
Best For	Large & Complex Apps	Small & Medium Apps
Learning Curve	Medium	Easy
Flexibility	Structured	Highly Flexible

★ 13. When to use What?

Use RTK when:

- Building large scale app.
- Need strict structure.
- Need RTK Query for API.
- Working in a big team.

VS

Use Zustand when:

- Building small / medium app.
- Need simple global state.
- Want less boilerplate.
- Like flexible approach.

★ 14. Quick Notes

- ✓ RTK → Structure + Convention + Powerful.
Good for long-term large projects.
- ✓ Zustand → Simplicity + Flexibility + Lightweight.
Good for quick and modern apps.

☑ Both are great!
Choose based on project size and team preference.



★ 15. One Line Summary

RTK

More boilerplate,
More power,
Better for big apps.

VS

Zustand

Less code,
More flexibility,
Better for simple apps.

★ RIGHT TOOL, RIGHT TIME, RIGHT PROJECT ★